📋 OpenSpec

IA como Copiloto · Parte 9 de 10

🌳 Nivel: avanzado · 📋 Spec-driven · 🧩 proposal / specs / design / tasks · 🔁 explore → propose → apply → archive

Sergi Garcia Barea · Noviembre 2026 · CC BY-SA 4.0

💥 El agente construye lo que entendió, no lo que querías

❌ Sin spec

- ✅ El plan vive en el chat: se pierde al cerrar la sesión
- ✅ «Hecho» significa cosas distintas para ti y para el agente
- ✅ Los requisitos se descubren a mitad del código
- ✅ Nadie puede revisar el plan: solo el diff final

✅ Con OpenSpec

- ✅ Un contrato de comportamiento en Markdown, versionado
- ✅ Humano y agente revisan el mismo plan antes del código
- ✅ Checklist de tareas: el progreso no vive en el chat
- ✅ Al archivar, las specs describen el sistema tal y como es

🎯 Idea central: **acordar qué se construye (y qué no)** antes de que el agente escriba la primera línea.

🔊

🧭 Qué es OpenSpec (y qué no es)

🛠️ Un CLI

Paquete `@fission-ai/openspec` sobre Node 22.12+. Sin claves de API y sin servidores MCP: `init`, `list`, `show`, `validate`, `archive`.

🧩 Unos workflows

Skills y comandos que se instalan en la carpeta de tu agente (`.agents/skills/`, `.opencode/commands/`, `.claude/`…). El agente los usa solo.

📁 Una carpeta

`openspec/` en la raíz del repo: specs del sistema, cambios en curso e histórico. Markdown plano que se lee sin la herramienta.

📌 Es **open source** (Fission-AI) y agnóstico: instala los mismos workflows en OpenCode, Claude Code, Codex, Cursor, Copilot, Antigravity y una veintena más.

🔊

🗂️ Anatomía de la carpeta openspec/

📂 UN CAMBIO EN CURSO

```
openspec/
├── config.yaml          ajustes y contexto
├── specs/               cómo es el sistema HOY
│   └── pagos/spec.md
└── changes/
    ├── add-reembolsos/  cambio EN CURSO
    │   ├── proposal.md   por qué y qué
    │   ├── specs/pagos/spec.md  delta
    │   ├── design.md     cómo (si hace falta)
    │   ├── tasks.md      checklist
    │   └── .openspec.yaml
    └── archive/        cambios TERMINADOS
        └── 2026-11-02-add-cupones/
```

📖 LAS DOS MITADES

- ✅ **specs/** = fuente de verdad del comportamiento actual, una capability por carpeta
- ✅ **changes/** = trabajo en marcha, una carpeta por cambio
- ✅ **changes/archive/** = histórico con fecha; nada se borra
- ✅ Todo se commitea con el código: la spec viaja en el mismo PR

🔊

⚙️ Instalación y arranque en 3 comandos

💻 TERMINAL

```
# 1) instala el CLI (Node 22.12+ mínimo)
npm install -g @fission-ai/openspec@latest
#    alternativa sin npm:
#    curl -fsSL
https://openspec.dev/cli/install.sh | sh

# 2) inicializa en la raíz del proyecto
cd mi-proyecto
openspec init
#    elige tus herramientas en el menú

# 3) refresca skills y comandos al actualizar
openspec update
```

OpenCode Freebuff · node ≥ 22.12

✅ QUÉ ESCRIBE INIT

- ✅ `openspec/config.yaml` con el schema `spec-driven`
- ✅ `openspec/specs/` y `openspec/changes/` vacías
- ✅ Skills `openspec-*` en `.agents/skills/`
- ✅ Comandos `/opsx-*` en la carpeta de tu agente
- ✅ Re-ejecutar `init` es seguro: refresca, no duplica

🔁 El ciclo: cinco pasos, siempre los mismos

1. **Explore** — piensas con el agente: lee el repo, pregunta, compara opciones. No escribe código ni artefactos.
2. **Propose** — redacta `proposal.md`, las specs delta, `design.md` y `tasks.md`. Se detiene en el plan.
3. **Review** — tú corriges el plan (a mano o pidiéndolo). Aquí se evita el 90 % del retrabajo.
4. **Apply** — el agente recorre `tasks.md` y marca casillas. Sesión nueva = retoma la primera sin marcar.
5. **Archive** — los requisitos entran en `specs/` y la carpeta pasa a `archive/` con fecha.

> 💡 Es el **Plan → Hacer** de la parte 2, pero con el plan convertido en artefactos del repo y una fase final que actualiza la documentación.

🔊

🗨️ Explore y Propose: de la idea al plan

🔍 EXPLORE · SIN ESCRIBIR NADA

```
"Explora cómo debería funcionar
el límite de peticiones de la API.
Lee el código, dime qué opciones
hay y qué preguntas faltan.
No escribas código ni ficheros."
```

📝 PROPOSE · EL PLAN COMPLETO

```
"Propón el cambio que acabamos
de discutir para añadir rate
limiting por IP y por usuario."
```

- ✅ **Explore** es modo pensar: el agente investiga, cuestiona y afina la idea; no toca el repo
- ✅ **Propose** escribe la carpeta del cambio: propuesta, requisitos testables, diseño y tareas
- ✅ Al terminar, el agente no ha modificado ni una línea de código

🔊

👀 Revisar el plan: el paso que ahorra el retrabajo

Orden	Fichero	La pregunta que responde	Qué corregir
1	`proposal.md`	¿Es el problema correcto, al tamaño correcto?	Alcance de más, «why» vago
2	`specs/`	¿Aceptaría estos requisitos como «hecho»?	Casos que faltan, requisitos no observables
3	`design.md`	¿Las decisiones técnicas y sus alternativas están claras?	Riesgos sin mitigación
4	`tasks.md`	¿Cubre las specs y nada más?	Tareas gigantes, verificación ausente

✍️ Son ficheros Markdown: edítalos tú o pide «corrige la spec, falta el caso sin autenticar». El plan es el contrato, no una sugerencia.

🔊

📐 Una spec de verdad: requisitos y escenarios

📄 OPENSPEC/CHANGES/ADD-REEMBOLSOS/SPECS/PAGOS/SPEC.MD

```
## ADDED Requirements
### Requirement: Reembolso dentro de plazo
El sistema SHALL permitir solicitar un
reembolso de un pedido entregado hace
menos de 30 días.

#### Scenario: Reembolso válido
- **WHEN** el cliente pulsa «Devolver»
  en un pedido entregado hace 12 días
- **THEN** se crea la solicitud y se
  responde con un 201

#### Scenario: Fuera de plazo
- **WHEN** el pedido se entregó hace 45 días
- **THEN** se responde 409 y se explica
  el motivo
```

🧠 REGLAS QUE VALIDA OPENSPEC VALIDATE

- ✅ Operaciones delta: **ADDED**, **MODIFIED**, **REMOVED**, **RENAMED**
- ✅ `### Requirement:` con SHALL / MUST, no «debería»
- ✅ `#### Scenario:` con WHEN y THEN (cuatro almohadillas, exactas)
- ✅ Un requisito = un comportamiento; el resto, escenarios
- ✅ Sin nombres de clases ni librerías: comportamiento, no implementación
- ✅ `validate --strict` añade el límite de 500 caracteres por requisito

🔊

🏗️ Apply y Archive: del plan al sistema

🏗️ Apply

- ✅ Lee la carpeta del cambio y trabaja `tasks.md` tarea a tarea
- ✅ Marca cada casilla al terminarla: no hay estado oculto
- ✅ Sesión nueva o contexto perdido: retoma la primera sin marcar
- ✅ «Aplica el cambio add-reembolsos» — y a revisar el diff

📦 Archive

- ✅ Fusiona el delta en `openspec/specs/`: añade o reemplaza requisitos
- ✅ Mueve la carpeta a `changes/archive/2026-11-02-add-reembolsos/`
- ✅ No borra nada: propuesta, diseño y tareas quedan como histórico
- ✅ Con todas las casillas marcadas, el agente propone archivar

🧪 OpenSpec ordena el proceso, **no comprueba** que el código cumple la spec: eso sigue siendo tuyo (y de los tests).

🔊

⌨️ CLI y comandos que usarás de verdad

Comando	Para qué
`openspec list`	Cambios en curso (nunca las specs)
`openspec list --specs`	Inventario de capabilities del proyecto
`openspec show <id>`	Leer propuesta, spec, diseño o tareas
`openspec validate <id>`	Validar formato del delta (`--strict` endurece)
`openspec archive <id>`	Fusionar en `specs/` y archivar con fecha
`openspec schemas`	Ver el flujo de artefactos del schema activo

🗣️ En el chat: **OpenCode** `/opsx-propose` · **Claude Code** `/opsx:propose` · **skills** `/openspec-propose`. Mismo workflow, distinta sintaxis según la herramienta.

⚖️ Cuándo sí, cuándo no y errores típicos

✅ Merece la pena

- ✅ Cambios con requisitos que alguien revisará
- ✅ Trabajo de varias sesiones o varias personas
- ✅ Proyectos heredados donde hay que entender el «ya existe»
- ✅ Cuando necesitas justificar por qué se construyó algo

⚠️ Errores típicos

- ✅ Spec que describe clases y librerías en vez de comportamiento
- ✅ Requisitos sin escenario o con THEN vago («funciona bien»)
- ✅ Un cambio-monstruo: mejor varios cambios pequeños encadenados
- ✅ Archivar sin revisar el diff ni correr los tests
- ✅ Usarlo para un typo o un prototipo que vas a tirar

🧭 Regla: si no puedes escribir el requisito en una frase comprobable, todavía no toca codificar.

🔊

🎓 Reto y resumen de la parte 9

📋 Mini-reto OpenSpec

Instala el CLI, inicializa tu proyecto y lleva un cambio real de explore a archive: propuesta, una spec con dos escenarios, tareas y validación

🤖 Prompt de arranque

> `"Explora el repo y propón con OpenSpec el cambio 'exportar notas a CSV'. No escribas código hasta que apruebe el plan."`

Siguiente (parte 10): **seguridad y confianza**. Recuerda el hilo: **plan en la parte 2** y **docs as code en la parte 6**.

CC BY-SA 4.0 · Sergi Garcia Barea · Noviembre 2026

🔊